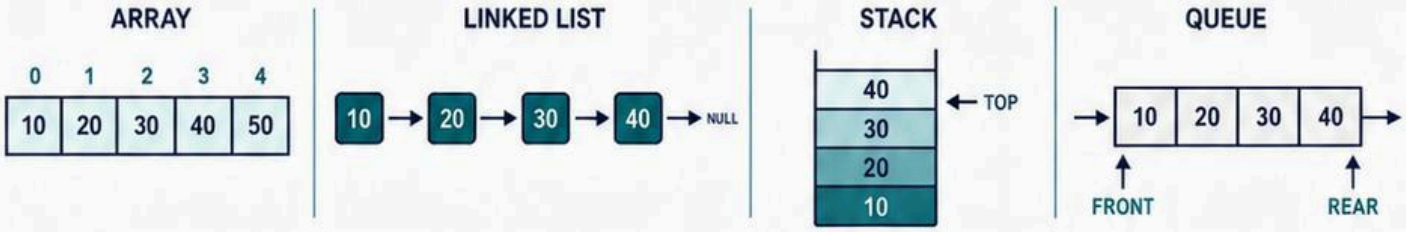
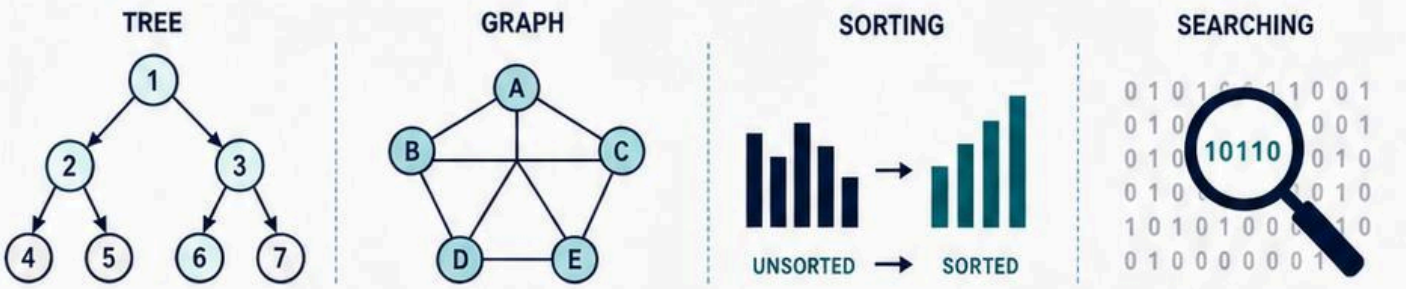
Mehran University of Engineering and Technology, Jamshoro

Department of Computer Systems Engineering

DATA STRUCTURES AND ALGORITHMS

CEP ENGINEERING REPORT





Student Name: Waqas Ali	**Instructor:** Dr. Ali Asghar Manjotho
Roll No: 25CS061	**Department:** Computer Systems Engg.
Section:	**University:** MUET, Jamshoro
Semester: Third	**Academic Year:** 2026

DESIGN IT. STRUCTURE IT. SOLVE IT.

</>

Design and Evaluation of an Efficient Context Budget Manager for LLM-Powered AI Agents

Complex Engineering Problem report · CS-251 Data Structures and Algorithms · Batch 25CS
Department of Computer Systems Engineering, MUET Jamshoro · Instructor: Dr. Ali Asghar Manjotho

Group member	Roll number	Main responsibility
Waqas Ali (group lead)	25CS061	Hybrid policy, hash map, lazy heap, packaging, live demo
Mehran Ali	25CS009	Loader and validation, three baselines, budget tests
Gaffar	25CS067	Labelled dataset and edge cases
Basit Azam	25CS113	Complexity analysis and scaling benchmark

Summary. An AI agent can only read a limited amount of text before it answers, and a retriever usually hands it far more than fits. This report describes **cbm**, a Context Budget Manager written in C++17 with our own binary heap, merge sort, linked list and hash map. We compared recency-first, relevance-first and relevance-per-token selection with a policy of our own, **hybrid fresh-cover**, which removes replaced notices and then repeatedly takes the passage offering the most new evidence per square root of its token cost.

On 32 labelled questions the hybrid covered 0.792, 0.908 and 1.000 of the needed facts at 256, 512 and 1,024 tokens, ahead of the best baseline at each budget (0.631, 0.842, 0.967), and never selected a replaced notice or a redundant copy. It went over budget zero times in 24,704 runs, as did every other policy. The price is real: it is roughly two and a half times slower than the baselines at normal query size and uses six to ten times their memory. Relevance-first also answered more facts on multi-source questions at 512 tokens. Answer quality was measured with an offline proxy, not a real language model.

1. Problem formulation 2
2. Assumptions and constraints 3
3. Stakeholder trade-offs 3
4. Alternatives and selection rationale 3
5. Data structure and algorithm design 4
6. Theoretical complexity 6
7. Experimental setup 6
8. Results 7
9. Answer-quality analysis 9
10. Engineering investigations 10
11. Verification and demonstration 10
12. Limitations 11
13. CEP attribute and outcome evidence 11
14. Project management and individual contributions 12
15. Conclusion 13
References 13
AI-use disclosure 13
Appendix A: Requirement traceability 14
Appendix B: Reproducing the results 14

1. Problem formulation

Picture a student asking the university's assistant for the last date of course registration. Before it answers, the agent receives course policies, notices, FAQ entries and pieces of earlier conversation, often several times more text than it may send to the model. Something has to decide what goes in. If that step is careless the agent can overflow its limit, lose the one fact that mattered, or quote a notice that was replaced last week and sound completely sure about it.

The brief asks one driving question: how should an agent represent, prioritise, select and order competing passages when memory, token capacity, relevance, time sensitivity and computing effort all limit what it can send? Our system begins after an existing retriever has supplied the candidates and ends with an ordered package that is guaranteed to fit. We did not train a model, build embeddings or write a vector database, since the brief places these outside the system boundary.

Capacity is calculated the way the brief defines it. From the total allocation B_{total} we reserve R_{fixed} for instructions and R_{answer} for the reply, which leaves

$$\mathbf{B_{CTX} = \max(0, B_{total} - R_{fixed} - R_{answer})} \quad \text{and every selection S must satisfy} \quad \mathbf{\Sigma_{i \in S}\, c_i \le B_{CTX}}$$

In our experiments R_{fixed} = 300 and R_{answer} = 500, so B_{total} values of 1,056, 1,312 and 1,824 give context budgets of 256, 512 and 1,024 tokens. The cost c_i is a nonnegative integer from the test fixture: body tokens plus 6 for the source label and separator. Costs in the main set run from 16 to 375 tokens, with a mean of 72.3 and a median of 54. No character-count estimate is used to claim that a limit was met.

2. Assumptions and constraints

We assume relevance scores in [0, 1] come from the retriever, freshness is an ISO timestamp, and a newer passage declares what it replaces through a `supersedes` link. Evidence tags of the form `key:value` are treated as trusted metadata. The last two assumptions are the weakest, and Section 12 returns to them.

The constraints of the brief were treated as fixed. Passages are indivisible, so an item goes in whole or not at all. Mandatory passages go first, and if they cannot all fit the system returns INFEASIBLE with a diagnostic instead of quietly dropping one. All four policies share one feasibility check and one tie chain (relevance descending, then timestamp descending, then passage id ascending), which makes the output deterministic for a fixed input. Every selected item keeps its passage id, source id, cost, priority and position. All data is local, and no network or model API is called.

Table 1. Targets we set for the solution and what was achieved (hybrid unless stated)

Target	Required	Achieved
Budget violations, all policies and runs	0	0 in 24,704 runs
Mean gold-evidence coverage at B_{CTX} = 512	≥ 0.85 and above every baseline	0.908 (best baseline 0.842)
Replaced passages selected while the newer one is available	0	0 at all budgets
Redundant copies, as a share of selected passages	≤ 5 %	0
Median selection time at n = 1,000 / 10,000	≤ 1 ms / ≤ 10 ms	0.139 ms / 5.05 ms
Auxiliary memory at n = 10,000	O(n), ≤ 2 MB	1.76 MB
Infeasible mandatory sets reported, never dropped	100 %	Q10 at 256 and 512, E05, E12, T04, T07

3. Stakeholder trade-offs

The three stakeholder groups in the brief want things that pull against each other, and each conflict changed the design.

Students against operators. Students want answers backed by evidence, which argues for more text; operators want token spending controlled. We stopped maximising raw relevance and started maximising *new* evidence per token, so the budget buys facts instead of repetition. The hybrid uses slightly less of the budget than relevance-first at 512 tokens (97.8 % against 98.5 %) because it would rather leave tokens unspent than add a copy that says nothing new. Relevance-first selected 82 redundant passages there and the hybrid none.

Policy officers against relevance. Officers need current, traceable rules to outrank old ones, but old notices and their forum reposts score 0.78 to 0.95 while the revision scores only 0.62 to 0.80. A policy that trusts relevance will happily choose the old text. We therefore exclude replaced passages and their copies before selection starts, and we keep the source on every item. Relevance-first selected 56 stale passages at 512 tokens; the hybrid selected none.

Quality against speed and memory. Tracking covered facts and replaced notices needs hash maps, which cost time and memory, and operators care about both. We accepted the extra cost because it stays small in absolute terms at real query sizes: 16 µs against 6 to 7 µs at about 41 candidates. At n = 10,000 the difference is more visible, at 5.05 ms and 1.76 MB against 0.84 ms and 0.23 MB for a heap baseline. Operators can still predict it, because memory stays linear in n.

4. Alternatives and selection rationale

Several design choices had a reasonable alternative. Where we could measure the difference we did, and Table 2 records the outcome.

Table 2. Alternatives considered

Decision	Options	Chosen	Reason
Ordering optional items	Merge sort all n and scan; or heapify and pop what is needed	Both kept, heap by default	Identical packages (T15). At n = 10,000: 127,267 comparisons and 3.22 ms for sorting, 19,287 and 0.88 ms for the heap.
Updating hybrid gains	Eager recompute each step; or lazy re-evaluation on pop	Lazy	Same output on 1,000 random instances (T16), with at most 309 re-insertions at n = 10,000.
Cost normalisation α	0 (gain only), 0.5 (per √token), 1 (per token)	0.5	Coverage at 256 tokens was 0.683, 0.792 and 0.750. α = 0 won on multi-source at 512 (0.875 against 0.825), so no value wins everywhere.
Stale passages	Penalty ×0.25; or exclusion	Exclusion	Same coverage, but the penalty let 2 stale passages in at 1,024 tokens. Officers need hard precedence.
Package store	Array with insertion; or linked list with list merge sort	Linked list	O(1) append while accepting and O(k log k) reorder with no second array. Since k is small, either would be fast enough.
Id and tag lookup	Linear search; or hash map	Hash map	Spreading stale marks would be $O(n^2)$ with search and is O(n) expected with hashing. Memory is the cost.
Exact optimisation	0/1 knapsack DP; or greedy	Greedy	The brief excludes advanced DP, and coverage is not additive. We therefore claim no optimality.

5. Data structure and algorithm design

5.1 Architecture

The system follows the separation suggested in the brief. A loader validates records, a policy engine selects, a validator recomputes the budget from the final ids, a packager orders the result and an agent adapter hands it on. These parts depend on each other more than the diagram suggests. The loader's duplicate-id rule is what makes a passage id safe to use as a hash key for stale detection, and the package order changes what a position-biased agent answers. The selector contains no model code, so a real language model could replace the offline agents without touching it.

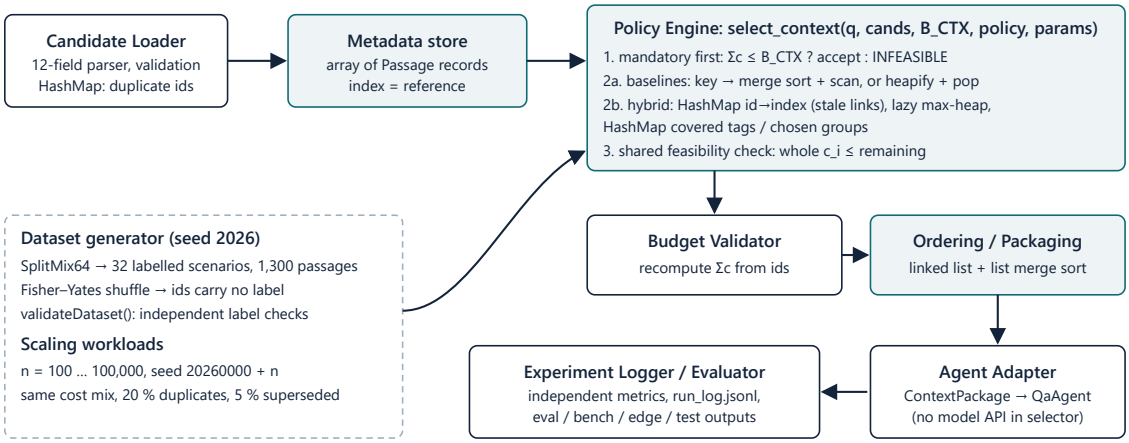


Figure 1. Components and data flow of the Context Budget Manager.

5.2 Structures we wrote ourselves

Table 3. Student-written structures and where they are used

Structure	Written by us	Used for	Cost
Binary heap	Floyd build, push, pop, top, invariant check, comparison counter	Heap engine of the baselines; lazy greedy of the hybrid	build O(n), push/pop O(log n)
Merge sort	Top-down, stable, one buffer, skips merge when halves are ordered	Sort engine of the baselines; mandatory order; medians	Θ(n log n), best Θ(n), space O(n)
Singly linked list	Tail pointer, in-place list merge sort with slow/fast split	Assembling and ordering the package	append O(1), sort O(k log k)
Hash map	Open addressing, linear probing, FNV-1a, growth at load 0.6	Duplicate ids, id to index, stale groups, covered tags, chosen groups	expected O(1), worst O(n)
SplitMix64, Fisher–Yates	Generator and shuffle	Reproducible dataset and workloads	O(1) per draw

`std::vector` and `std::string` serve only as storage. Standard streams handle I/O, `steady_clock` handles timing, and `std::pow` handles the exponent. We used no `std::sort`, `priority_queue`, `make_heap`, `unordered_map`, `map` or `list`. The plotting script uses only Python's standard library.

5.3 The four policies

The three baselines differ only in their key: timestamp for recency-first, relevance for relevance-first, and relevance ÷ c_i for relevance-per-token. Each places mandatory items, then walks the rest in key order and accepts whatever still fits. Each has a merge-sort engine and a heap engine, and the two always produced the same package (T15, 3,000 runs).

The hybrid responds to freshness and evidence coverage together, because the two interact: removing stale items frees budget for coverage, and discounting copies stops one long group crowding out other sources. A passage's weight combines its relevance with its source reliability (A 1.0, B 0.85, C 0.7). Its gain depends on what has already been chosen, so its priority falls as facts get covered.

```
w(i)        = relevance(i) × reliabilityWeight(i)
gain(i | S) = w(i) × (λ + (1−λ) × newTags(i,S)/|tags(i)|) × (dupPenalty if group(i) already in S)
priority(i) = gain(i | S) / max(c_i, 1)^α                    // α = 0.5, λ = 0.2, dupPenalty = 0.3

HYBRID_SELECT(cands, B_CTX):
  ids ← HashMap passage_id → index
  mark every passage replaced by a candidate, and every copy of it, as stale
  if Σ cost(mandatory) > B_CTX: return INFEASIBLE(diagnostic)
  accept every mandatory item; mark its tags covered and its group chosen
  items ← { (priority(i), i) : optional, not stale, c_i ≤ B_CTX }   // others omitted with a reason
  heap ← FLOYD_BUILD(items)
  while heap not empty and remaining ≥ minCost:
      (key, i) ← heap.pop()
      if c_i > remaining:              omit i "insufficient capacity"; continue
      if group chosen and no new tag:  omit i "duplicate, no new evidence"; continue
      p ← priority(i)                                        // true value now
      if p < key and heap.top() beats (p, i): heap.push((p, i)); continue
      accept i; remaining −= c_i; mark tags covered, group chosen
  package ← LINKED_LIST(accepted); LIST_MERGE_SORT(package, order rule)
```

5.4 Invariants and budget safety

- **Heap order.** Parent ≥ child after every sift, checked with `isHeap()` under 20,000 random operations (T17).
- **Stored keys are upper bounds.** As S grows, newTags can only fall and a group can only become chosen, so a gain never rises. A popped item that still beats the top is therefore truly the best, which is why the lazy result equals eager greedy (T16: 1,000 instances, 1,442 re-insertions, 0 mismatches).
- **Partition.** Every candidate leaves through exactly one accept or one omit, so selected and omitted sets are disjoint and complete (T14).
- **No stale optional item.** Stale passages are removed before the heap is built (T13, E08).

Why the budget can never be exceeded. Let r be the remaining capacity, starting at $B_{CTX} \geq 0$. If mandatory items exceed B_{CTX}, nothing is accepted and the status is INFEASIBLE. Otherwise every acceptance in every policy goes through one function that accepts i only when $c_i \leq r$ and then sets $r \leftarrow r - c_i$. By induction $r \geq 0$ after every step, so $\Sigma\ c(S) = B_{CTX} - r \leq B_{CTX}$. A separate validator also recomputes the total from the final ids and would raise BUDGET_VIOLATION; it never fired.

The hybrid is a heuristic. Covering the most tags under a token budget is a budgeted maximum-coverage problem, which is NP-hard, and plain cost-scaled greedy has no constant-factor guarantee without the extra comparison described by Khuller et al. [5], which we did not implement.

6. Theoretical complexity

Below, n is the number of candidates, m mandatory items, k selected items, p heap pops, r lazy re-insertions, T tags per passage (at most 4 here) and L the id length. Auxiliary space excludes the input and the output lists.

Table 4. Time and auxiliary space

Operation	Best	Average	Worst	Auxiliary space
Load and validate N records	O(N·L) expected		$O(N^2 \cdot L)$ if all ids collide	O(N) slots
Baseline, sort engine	Θ(n), presorted	Θ(n log n)	Θ(n log n)	Θ(n)
Baseline, heap engine	O(n + k log n)	O(n + p log n)	O(n log n)	Θ(n)
Hybrid, lazy greedy	O(n·L + k(log n + T))	O(n·L + (p + r)(log n + T)) expected	O(n·L + n·k(log n + T))	Θ(n), 4 hash maps, weights, heap
Packaging	Θ(k log k) comparisons, O(k) append			O(k) + O(log k) stack
Budget validation	Θ(k)			O(1)

Sorting pays Θ(n log n) however few items end up selected. The heap pays O(n) to build and a log n factor only for items it actually pops, so it is close to linear when k is much smaller than n, which is the normal case. As the budget grows, p approaches n and the heap's cost approaches the sort's. The hybrid adds an O(n) hashing pass with a large constant, because it hashes and copies strings. Section 8.4 tests each of these predictions.

7. Experimental setup

Main dataset. `cbm gen` (seed 2026) creates 32 labelled questions with 1,300 candidate passages: 10 time-sensitive policy questions, 10 multi-source questions, 10 conversation-memory questions and 2 with no answer in the data. Each policy scenario contains a revised notice, the old notice with three reposts that score higher, and irrelevant recent notices. Multi-source scenarios split four facts over four sources and add one long overview, near-duplicates and a contradicting forum post. The set holds 90 gold tags, 20 supersede links and 11 mandatory passages.

Labels. The generator writes each tag in the same step as the sentence stating it, and gold is the set of values stated by current official or FAQ passages. Copies carry the same tag and coverage is a set union, so repetition never counts as independent evidence. Passages are shuffled before ids are assigned, so an id reveals nothing. A separate code path (`cbm validate`) then checks that every tag value appears in its passage text, every gold tag is stated by a passage nothing replaces, and every supersede link points to an older passage. It reported 0 problems. Questions with no gold facts are left out of the coverage mean and scored only on whether the agent abstains.

MUET document set. We also ran the system on the corpus from our PBL project: 31 documents of the Department of Computer Systems Engineering, 155 passages and 20 hand-labelled questions. Every question receives all 155 passages, giving 3,100 records. Relevance here is BM25 scaled to the best passage, and cost is an estimate, ceil(1.3 × words) + 6, since no tokenizer was available. Three revised notices were linked to the originals they replace: attendance from 70 % to 75 %, exam dates, and registration from 10 to 17 September. These rules were mostly written by the team for testing, and only document D24 restates a real brief.

Scaling workloads. Synthetic sets of n = 100, 1,000, 10,000 and 100,000 use the same cost mix, seed 20260000 + n, 20 % duplicates and 5 % superseded passages.

Protocol. The main comparison runs 32 questions × 3 budgets × 4 policies = 384 runs on identical input, with hybrid parameters fixed before evaluation. Timing uses `std::chrono::steady_clock` around the selector, with 1 warm-up and 5 timed runs per configuration (9 for scaling), and we report medians with ranges. The machine was an Intel Core i7-8650U (4 cores, 8 threads, 1.9 GHz base) with 15.8 GB RAM on Windows 11 Pro, compiled by TDM-GCC 10.3.0 with `g++ -std=c++17 -O2 -Wall -Wextra` and 0 warnings. The CPU was 43 % busy with other work during the runs, which explains the wide ranges. All metrics are recomputed from the selected ids by an evaluator separate from the selector.

8. Results

8.1 Main comparison

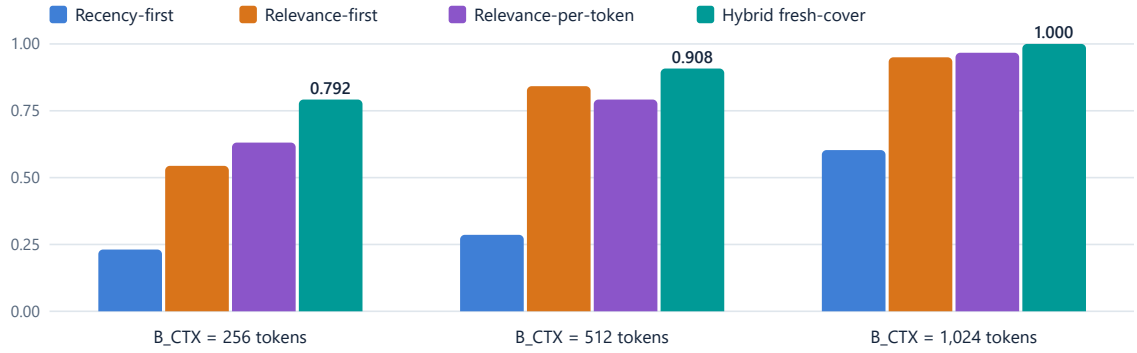


Figure 2. Mean gold-evidence coverage over the 30 answerable questions. Values are labelled for the hybrid; Table 5 has the rest.

Table 5. All four policies on the 32 labelled questions

B_{CTX}	Policy	% B_{CTX} used	Coverage	Redundant	Sources	Stale (queries)	Median µs [range]	Aux. bytes
256	Recency	97.3	0.231	11	2.61	0 (0)	6.40 [3.5–12.7]	1,904
	Relevance	96.9	0.544	76	3.23	42 (19)	6.75 [3.9–14.7]	1,939
	Per-token	95.6	0.631	67	3.55	42 (19)	7.35 [3.3–16.4]	2,061
	Hybrid	95.6	**0.792**	**0**	**3.65**	**0 (0)**	16.30 [3.3–38.4]	18,149
512	Recency	98.3	0.286	34	3.55	0 (0)	6.40 [2.8–14.2]	2,180
	Relevance	98.5	0.842	82	3.81	56 (19)	6.45 [2.7–15.6]	2,166
	Per-token	97.6	0.792	109	5.03	48 (19)	7.00 [2.2–12.2]	2,538
	Hybrid	97.8	**0.908**	**0**	4.97	**0 (0)**	15.85 [2.7–18.8]	18,530
1,024	Recency	99.1	0.603	84	5.06	4 (2)	6.80 [5.0–14.4]	2,870
	Relevance	99.3	0.950	119	5.44	60 (20)	6.90 [4.5–13.0]	2,820
	Per-token	98.3	0.967	152	5.91	57 (20)	7.40 [5.5–8.9]	3,286
	Hybrid	98.8	**1.000**	**0**	5.84	**0 (0)**	16.95 [11.3–20.4]	19,688

No policy violated the budget in any of the 384 runs. Question Q10 needs two mandatory passages totalling 620 tokens, so it is INFEASIBLE at 256 and 512 for every policy and counts as zero coverage. The hybrid's lead is largest when the budget is tight: 0.161 above the best baseline at 256 tokens, 0.066 at 512 and 0.033 at 1,024. With plenty of room, even a careless policy eventually picks up most facts, so what separates the policies at that point is mostly the stale and redundant content they bring along.

The cost is plain in the last two columns. The hybrid takes about 16 µs where the baselines take 6 to 7 µs, and it uses six to ten times their auxiliary memory. Its source diversity is also not always the best: relevance-per-token reached 5.03 sources at 512 tokens against the hybrid's 4.97, and 5.91 against 5.84 at 1,024.

8.2 Results by question type

Recency-first is clearly the weakest for memory and policy questions, because key facts are often older than the chatter around them. Multi-source questions at 512 tokens are the one place the hybrid does not lead on coverage: it ties relevance-first at 0.825, although it gets there by a different route, using four short sources and no copies, while relevance-first takes the long overview.

8.3 Mehran University documents

Table 6. MUET set: 20 questions, 155 candidates each, 320 runs, 0 violations

B_{CTX}	Measure	Recency	Relevance	Per-token	Hybrid
128	Coverage	0.000	**0.941**	0.853	0.912
	Stale passages selected	7	10	17	**0**
512	Coverage	0.088	0.971	0.971	**1.000**
	Stale passages selected	100	53	59	**0**
512	Authority agent, correct of 22 (contradictory)	2 (0)	21 (1)	21 (1)	**22 (0)**

This set shows the problem in a form anyone at MUET would recognise. Ask for the registration deadline with only 128 tokens and relevance-per-token answers 10 September from the replaced notice. Relevance-first gets 17 September but has both notices in its context and flags that the sources disagree. The hybrid answers 17 September with no conflict. It was the only policy that never selected a replaced notice in this set and never gave a contradictory answer. It was not perfect, though: at 128 tokens it skipped the 45-token revised attendance notice for cheaper passages and missed both attendance facts on question M09, which is why its coverage trails relevance-first at that budget. On this set it took 30 to 35 µs against 12 to 16 µs for the baselines.

8.4 Scaling and memory

Table 7. Median µs per selection at B_{CTX} = 1,024 (9 timed batches after a warm-up)

Policy / engine	n = 100	n = 1,000	n = 10,000 [range]	n = 100,000
Recency / merge sort	12.16	107.85	3,222 [2,107–4,136]	47,231
Recency / heap	8.92	27.93	881 [795–984]	16,600
Hybrid / lazy heap	20.89	139.32	5,046 [3,634–6,960]	64,687

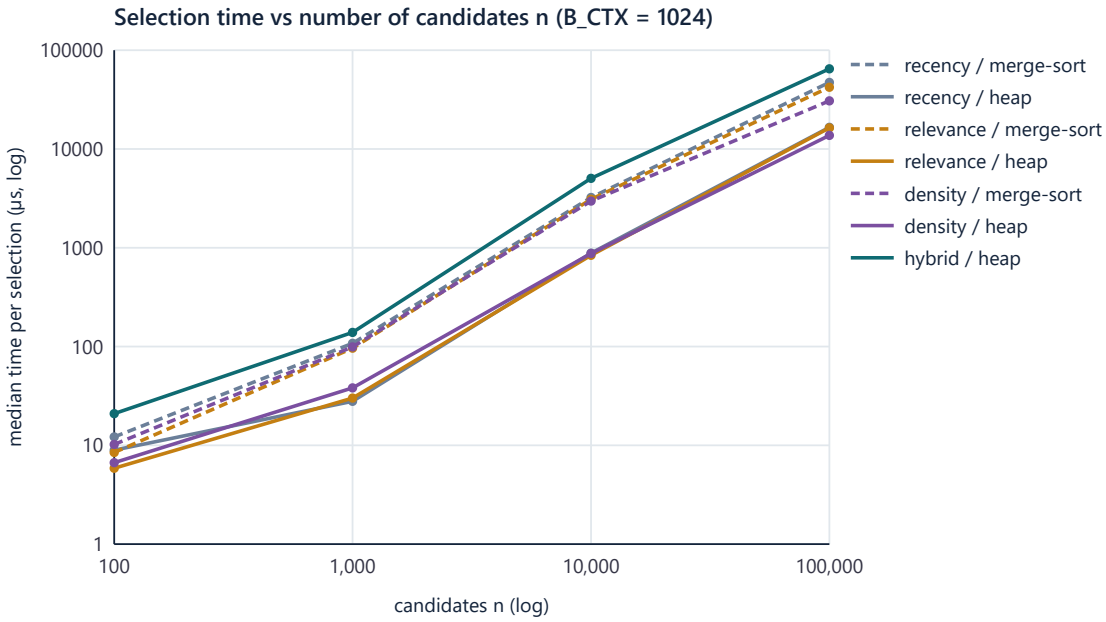


Figure 3. Selection time against the number of candidates.

Table 8. Operation counts and memory

Measure	n = 10,000	n = 100,000	Theory
Merge-sort comparisons (recency)	127,267	1,605,791	n $\log_2$ n = 132,877 / 1,660,964
Heap comparisons with Floyd build (recency)	19,287	188,469	build ≤ 2n, plus p log n
Hybrid: hashing pass / total µs	3,854 / 5,046 (76 %)	48,495 / 64,687 (75 %)	O(n·L) dominates
Baseline sort: preparation / total µs	245 / 3,222 (8 %)	3,817 / 47,231 (8 %)	n log n dominates
Auxiliary KB: sort / heap / hybrid	391 / 235 / 1,763	3,907 / 2,345 / 14,870	all Θ(n)

The operation counts match theory closely. Merge sort used about 0.96 × n $\log_2$ n comparisons at both sizes, and its count stayed at 127,266 for the relevance-per-token key at every budget from 256 to 65,536. Heap comparisons stayed near 2n and grew only with pops, from 19,225 at 12 pops to 63,586 at 1,838. The heap was 2.2 to 3.9 times faster than sorting at n ≥ 1,000, and the gap closed as predicted when k grew: at B_{CTX} = 65,536 the heap took 1,908 µs against 3,557 µs.

Time grew faster than the comparison counts between n = 1,000 and 10,000, about ×30 for sorting against ×13 for n log n. The most likely explanation is that the records stop fitting in the CPU caches at that size, on top of the load on the machine, although we did not measure cache misses. From 10,000 to 100,000 sorting grew ×14.7 against ×12.5 predicted. Three separate benchmark runs at n = 10,000 gave recency/heap medians of 878, 868 and 881 µs and hybrid medians of 4,515, 4,657 and 5,046 µs, so the comparison between policies is repeatable even though the ranges are wide.

8.5 Sensitivity of the hybrid

Table 9. Changing one design choice at a time

Variant	Coverage 256 / 512 / 1,024	Multi-source at 512	Stale at 1,024	Redundant at 1,024
Default: α = 0.5, exclude stale, drop copies	0.792 / 0.908 / 1.000	0.825	0	0
α = 0, no cost normalisation	0.683 / 0.892 / 0.975	**0.875**	0	0
α = 1, full per-token	0.750 / 0.900 / 1.000	0.800	0	0
Stale penalty ×0.25 instead of exclusion	0.792 / 0.908 / 1.000	0.825	2	0
Keep redundant copies	0.792 / 0.908 / 1.000	0.825	0	5
No coverage or redundancy terms (λ = 1, dup = 1)	0.717 / 0.825 / 0.983	0.575	0	114

The coverage and redundancy terms contribute the most, adding 0.083 at 512 tokens. Hard exclusion of stale items changes nothing until the budget is large enough for a penalised item to creep in. No single α wins at every budget, so 0.5 is a compromise we chose, not a best value we can prove.

9. Answer-quality analysis

We did not call a language model. Two deterministic extractive agents read only the context package. **First-mention** is position-biased and uses the first passage stating each fact; **authority** ignores order and prefers the most reliable, then newest, source. For each asked fact, the answer is correct if it gives the gold value, contradictory if it gives another value, and missing if it gives none. Because these agents only repeat what the context says, unsupported claims are zero by construction. This is a **proxy for answerability, not observed LLM accuracy**, and it should be read that way.

Table 10. Answer-quality proxy at B_{CTX} = 512 (first-mention · authority)

Questions	Recency	Relevance	Per-token	Hybrid
Policy Q01–Q04, facts correct	4/8 · 4/8	6/8 · 6/8	7/8 · 7/8	**8/8 · 8/8**
Multi-source Q11–Q14	3/16 · 4/16	**14/16 · 14/16**	6/16 · 9/16	11/16 · 13/16
Memory Q21–Q24	4/12 · 4/12	8/12 · 12/12	10/12 · 12/12	**12/12 · 12/12**
12 fixed questions, total correct	11/36 · 12/36	28/36 · 32/36	23/36 · 28/36	**31/36 · 33/36**
Contradictory claims on the 12	4 · 3	4 · 0	6 · 1	**2 · 0**
All 30 answerable, correct of 90	22 · 24	67 · 77	55 · 68	**75 · 81**

The hybrid gave the most correct answers overall at all three budgets (84 and 90 of 90 at 1,024 tokens; 68 and 69 at 256), and all policies abstained correctly on both no-gold questions. **It lost in one place that matters.** On multi-source questions at 512 tokens, relevance-first answered 14 of 16 facts against the hybrid's 11 to 13, because the long overview it keeps states three facts at once. The hybrid's remaining contradictions, 5 to 6 across the full set with the first-mention agent, come from passages with no supersede link: an unverified forum post and a memory summary still holding an old value. No amount of careful selection fixes metadata that was never recorded.

10. Engineering investigations

When is a long, relevant passage better than several short ones? When it fits with room to spare and states several needed facts that are not available more cheaply. The multi-source overview (260 to 380 tokens) never fits at 256. At 512, taking it gave relevance-first 0.825 and the α = 0 hybrid 0.875, against 0.825 for four short sources in the default hybrid and 0.525 for relevance-per-token. At 1,024 the short sources won outright: hybrid 1.000 against 0.900 for relevance-first, which spent its budget on the overview and copies.

When does newest-first lose evidence, and when does relevance-first include stale content? Newest-first fails when key facts are older than the chatter around them. Memory coverage was only 0.067, 0.133 and 0.533, and policy coverage stayed at 0.450 up to 512 because the 8 newest notices were irrelevant. Relevance-first includes stale content whenever old text outscores its revision, selecting 42, 56 and 60 stale passages across 19 to 20 questions.

Can relevance-per-token lose crucial evidence despite high utility? Yes, in 36 question and budget cases. On Q01 at 256 tokens it selected 6 passages with total relevance 4.114, including three cheap reposts of the old deadline, and missed the form name (coverage 1/2). The hybrid selected 5 passages with total relevance of only 3.157 and covered 2/2. Cheap copies look efficient but add no new evidence.

What dominates the runtime? For the sort engine it is the merge sort itself, 92 % of the time and independent of k. For the heap engine it is the O(n) build, then p log n as k grows. For the hybrid it is the hashing pass at 75 to 76 %, not the heap, with re-insertions never above 309. If the hybrid ever needs to be faster, the id handling is the place to start.

How should the system behave in awkward cases? If crucial items do not fit, it returns INFEASIBLE and names each mandatory id, its cost, the total and the budget (E05: `E05-P1(150) + E05-P2(180) need 330 tokens but B_CTX = 256`). An optional item larger than the budget is simply omitted while the rest are still chosen (E04). Replaced versions are excluded; other contradictions are kept with their source, and the authority agent prefers the more reliable one and flags the disagreement. Ties follow the fixed chain (E06, T09). In a near-duplicate group the hybrid keeps a copy only if it adds a new tag (E07: 2 passages and 2/2 coverage, against 5 copies and 1/2). A repeated copy could still be useful as corroboration from a separate source, but it would have to be scored as that, and we did not model it.

Does the order of the context change the answers? It depends on the agent. With the same selection, the authority agent's result did not change under any order (relevance-first at 512: 77/90). The first-mention agent did change: relevance-first scored 58/90 oldest-first, 67/90 by priority and 76/90 newest-first. The hybrid moved less, from 75 to 71, because its stale passages are already gone. Selection quality and order effects therefore need separate measurement, and order effects should be re-measured with a real model [6].

11. Verification and demonstration

`cbm test` runs 29 automated tests and all pass. They cover empty input, zero budgets, malformed records, duplicate ids, ties, heap and hash-map behaviour, and determinism. Three are property tests that give the strongest evidence: T14 checks budget safety and the partition rule on 24,000 random runs, T15 compares the heap and sort engines over 3,000 runs, and T16 compares lazy and eager greedy on 1,000 instances. With the 384 evaluation runs and 320 MUET runs, that makes 24,704 runs with no violation. The loader rejected all 15 malformed records in the fixture, each with a line number and a specific reason, such as `negative token cost '-20'` or `duplicate passage id M01-P1 in M01 (first seen on line 2)`.

The edge cases required by the brief are in `results/edge_cases.txt` as E01 to E13. Two of them are worth noting because the hybrid gains nothing there. In E09, evidence split over three sources gives 1/3 for all four policies, because equally relevant distractors win. In E11 at 1,024 tokens, it ties recency-first and relevance-first at 4/4. A live run in the viva follows `cbm select`, `cbm compare`, an infeasible query on Q10, and `cbm bench`, and the interactive menu (`cep/run_demo.bat`) provides the same steps.

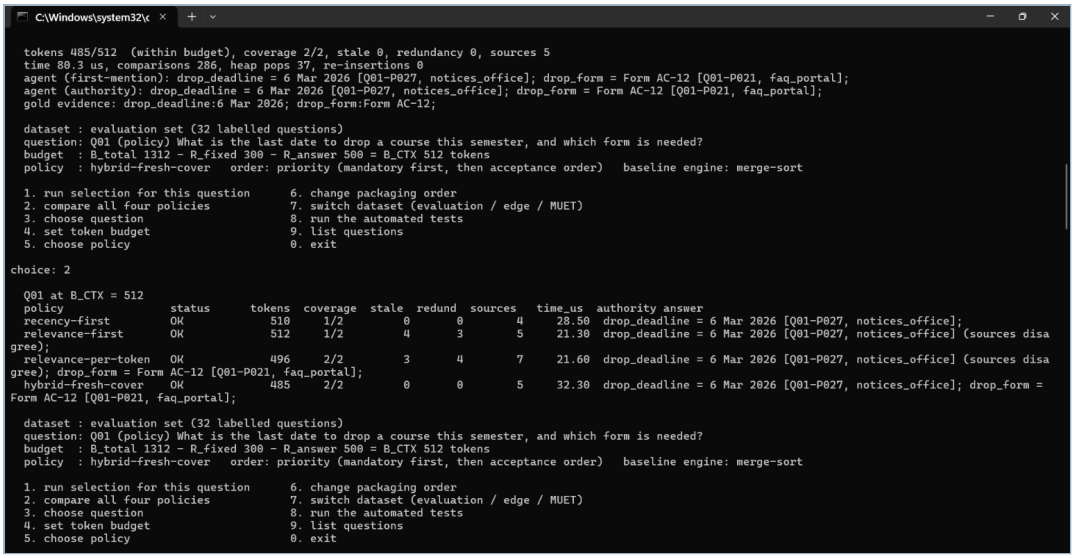


Figure 4. Coverage 2/2 with both agents answering 6 Mar 2026 and Form AC-12, followed by the four-policy comparison. Only the hybrid covers 2/2 with no stale and no redundant passages.

12. Limitations

- **Freshness depends on explicit links.** A contradiction with no `supersedes` link goes undetected, and this causes the hybrid's remaining contradictory answers. A per-key "newest wins" rule over evidence tags is the obvious next step.
- **Coverage depends on the retriever.** The hybrid cannot tell needed facts from distractors with the same relevance (E09).
- **No optimality guarantee.** $\alpha = 0.5$ under-weights a long multi-fact passage at mid budgets and can skip a needed longer passage at small ones (M09 at 128 tokens).
- **The answer measure is a proxy.** Two simple agents stand in for a real model. A check with an LLM has not been done, so nothing here shows how a real model would answer.
- **Most data is synthetic or team-written.** Absolute coverage numbers depend on how relevance and cost were designed, so the comparison between policies is the real finding. MUET costs are word-count estimates.
- **Noisy timing.** Benchmarks ran on a laptop under 43 % load. Medians, ranges and repeated runs reduce this but do not remove it.

We know of no crashes or failing tests at the time of writing.

13. CEP attribute and outcome evidence

Table 11. Where each attribute and outcome is demonstrated

Attribute / outcome	Evidence	Section
WP1 Depth of knowledge	Own heap, merge sort, linked-list sort and hash map; invariants; lazy-greedy correctness argument; budget-safety proof	5.2, 5.4
WP2 Conflicting requirements	Freshness against relevance, coverage against tokens, quality against latency and memory, each quantified	3, 8.1
WP3 Depth of analysis	No policy dominates: relevance-first wins multi-source answers at 512, $\alpha = 0$ beats the default there, and E09 fails for all	8.5, 9, 11
WP6 Stakeholder needs	Each stakeholder conflict tied to a design decision and its measured cost	3
WP7 Interdependence	Loader and hashing, metadata and selection, ordering and agent answers	5.1, 10
CLO-2 (C3, PLO-2)	Problem formulation, alternatives, justified selection and ordering algorithms	1, 4, 5
CLO-3 (C4, PLO-3)	Best, average and worst-case and auxiliary-space analysis compared with measurements	6, 8.4

14. Project management and individual contributions

Table 12. Five-week plan and responsibilities

Week	Work package	Owner
1	Requirements, stakeholders, budget model, dataset schema, risk register	Waqas Ali (lead), Mehran Ali
2	Loader and validation; three baselines with both engines; budget tests	Mehran Ali
3	Hybrid policy, hash map, lazy heap, linked-list packaging, invariants	Waqas Ali
4	Labelled dataset, edge cases, complexity analysis, scaling benchmark, agent proxy	Gaffar (dataset, edge cases), Basit Azam (complexity, benchmark)
5	Comparison, investigations, report, demo, viva	All four; Waqas Ali leads the demo

Table 13. Individual contributions

Member	Designed	Implemented	Validated	Documented
Waqas Ali	Requirements and budget model (with Mehran Ali); hybrid policy	Hash map, lazy heap selection, linked-list packaging	Invariants; T12–T16, T19–T22	Hybrid design and proof; demo
Mehran Ali	Requirements and dataset schema (with Waqas Ali)	Loader, three baselines, sort and heap engines	Budget and metadata tests, T01–T15	Validation rules and baselines
Gaffar	Scenario categories and labelling method	Dataset generator, edge-case fixtures	Label checks, E01–E13	Dataset and edge-case results
Basit Azam	Benchmark method	Scaling workloads, benchmark, plots	Repeated runs, theory against measurement	Complexity analysis and scaling results

All four members took part in the policy comparison, the engineering investigations and this report, and each explains the part they own in the viva.

Table 14. Decisions that changed the work

What we observed	What we changed	Effect
The hybrid filled leftover budget with copies of a covered group, failing T12	Added a rule to omit copies adding no new tag	Redundancy 0; T12 and T16 pass
All memory scenarios had chatter only after key turns, so recency scored 0.0, an artefact of our data	Generator now places 0 to 27 trailing chatter turns across Q21–Q30	Recency memory coverage 0.067 / 0.133 / 0.533
Benchmark ranges were wide on a busy machine	Repeated with 9 repetitions on the final binary, keeping earlier runs	Three runs kept and compared (8.4)
Needed evidence on what dominates runtime	Added a preparation-phase timer to the metrics	Hybrid: 75–76 % is hashing

Table 15. Risk log

Risk	Mitigation	Status
Budget overflow	One shared check, separate validator, randomised property test	0 violations
Silent loss of mandatory evidence	INFEASIBLE with diagnostic, nothing partial	T04, T07, Q10
Lazy greedy differing from intended greedy	Eager oracle on 1,000 random instances	T16, 0 mismatches
Label leakage or bad gold data	Shuffle before ids; separate validator; text-against-tag check	0 problems
Timing noise	Warm-up, repetitions, medians and ranges, repeated runs, operation counts	Ranges still wide
Results differing across compilers	Own SplitMix64 in place of standard distributions	T27, T28

Retrospective. The property tests served us best, catching design gaps early and giving strong evidence of budget safety. The first memory dataset went badly: it was too uniform and handed recency a misleading score of zero until we fixed the generator. Next time we would fix evaluation parameters before generating data, run benchmarks on an idle machine, and plan the real-model check from the start.

15. Conclusion

Returning to the driving question: an agent should treat each passage as an indivisible item with a cost and known evidence, settle mandatory items first, remove what has been replaced, and then choose by the new evidence each passage adds relative to its size. Our measurements support that approach. The hybrid led on coverage at every budget on the main set, kept out stale notices and copies, and never broke the budget.

It is not a universal winner. It costs more time and memory, it can lose to a long passage on multi-source questions, it can skip a needed notice at a very small budget, and it trusts metadata that a real system may not have. The clearest lesson for us was that the difficulty lies less in fitting text into a limit than in deciding which text deserves the space. The next real tests are a per-key freshness rule and an evaluation with an actual language model.

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
2. R. W. Floyd, "Algorithm 245: Treesort 3," *Communications of the ACM*, 7(12), 1964.
3. M. Minoux, "Accelerated greedy algorithms for maximizing submodular set functions," *Optimization Techniques*, LNCS 7, Springer, 1978.
4. G. L. Nemhauser, L. A. Wolsey, M. L. Fisher, "An analysis of approximations for maximizing submodular set functions I," *Mathematical Programming*, 14, 1978.
5. S. Khuller, A. Moss, J. Naor, "The budgeted maximum coverage problem," *Information Processing Letters*, 70(1), 1999.
6. N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the ACL*, 2024.
7. G. L. Steele, D. Lea, C. H. Flood, "Fast splittable pseudorandom number generators," OOPSLA 2014.

AI-use disclosure

The C++ code, the synthetic dataset generator, the experiments and this report were produced with the help of an AI coding assistant (Claude, Anthropic) under the team's direction. Every number in this report comes from running the submitted code, with the outputs kept in `cep/results/`. Each member must be able to explain the code they present in the viva.

Appendix A: Requirement traceability

Requirement	Implementation	Evidence
FR-1 Representation	`struct Passage` with all fields from the brief	passage.h
FR-2 Ingestion and validation	12-field parser, per-field checks, hash-map duplicate check, line-numbered errors	T01, T03, T10, T11
FR-3 Baselines	Three policies, each with sort and heap engines, same checks and ties	T15, Table 5
FR-4 Proposed policy	Hybrid fresh-cover	Section 5.3, Tables 5–6
FR-5 Hard budget	Mandatory first or INFEASIBLE; capacity check before every item	T04, T07, T08, T14, E11
FR-6 Redundancy and coverage	Duplicate group and tags tracked; copies discounted ×0.3 and dropped if they add nothing	T12, E07
FR-7 Source and freshness	Stale exclusion, ×0.25 penalty as sensitivity, reliability weights, provenance kept	T13, E08, Table 9
FR-8 Ordering and packaging	Linked-list package with three order rules; used and unspent tokens, selected ids, omitted ids with reasons	T22, T23, sample_package.txt
FR-9 Agent interface	`buildPackage → ContextPackage → QaAgent::answer`, no model code in the selector	T23, T24
FR-10 Observability	JSON run log with policy, parameters, n, seed, time, counts, memory, status, ids (384 lines)	run_log.jsonl

Appendix B: Reproducing the results

All commands run from the `cep/` folder with no paid service or network access.

```
run_demo.bat                    interactive menu (or: cbm menu)
build.bat                       g++ -std=c++17 -O2 -Wall -Wextra
cbm gen        → data/scenarios.txt, data/passages.txt (seed 2026)
cbm validate   → results/dataset_validation.txt
cbm test       → results/tests.txt
cbm edge       → results/edge_cases.txt
python scripts/muet_dataset.py → data/muet/
cbm muet       → results/muet_summary.txt
cbm eval       → results/eval_summary.txt, answer_quality.txt, order_experiment.txt, sensitivity.txt,
run_log.jsonl
cbm bench --reps 9 → results/bench_scaling.csv, bench_summary.txt
python scripts/plot_scaling.py → results/scaling_n.svg, scaling_k.svg
```